

MQTT Lab Manual

Publisher / Subscriber with Paho & Mosquitto

1. Requirements

Before starting the lab, make sure the following software and packages are installed on your machine.

1.1 System Requirements

- Python 3.8 or higher
- Docker Desktop (with Docker Compose v2 support)
- A terminal / command-line interface

1.2 Python Dependencies

Install the Paho MQTT client library using pip:

```
pip install paho-mqtt
```

1.3 Docker Compose

Docker Compose is used to run the Mosquitto broker. Verify it is available:

```
docker compose version
```

You should see output similar to: Docker Compose version v2.x.x

2. Setup

This section explains how to start the Mosquitto MQTT broker using Docker Compose with the dedicated mosquitto profile.

2.1 Docker Compose File

Create (or verify) a docker-compose.yml file at the root of your project with the following content:

```
services:  
  mosquitto:
```

image: eclipse-mosquitto:latest
profiles: - mosquitto
ports: - "1883:1883"
volumes: - ./mosquitto/config:/mosquitto/config - ./mosquitto/data:/mosquitto/data - ./mosquitto/log:/mosquitto/log

Also create a minimal Mosquitto configuration file at `./mosquitto/config/mosquitto.conf`:

listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log

2.2 Starting the Broker

Run the following command from the directory containing your `docker-compose.yml` file to start Mosquitto using the `mosquitto` profile:

<code>docker compose --profile mosquitto up -d</code>

To verify the broker is running:

<code>docker compose --profile mosquitto ps</code>
--

You should see the `mosquitto` service listed with status `running`.

2.3 Stopping the Broker

When you are done with the lab, stop the broker with:

<code>docker compose --profile mosquitto down</code>
--

2.4 Configuration Values

Once the broker is running, note the following connection values to use in both scripts:

Parameter	Value	Description
-----------	-------	-------------

BROKER	localhost	Hostname of the broker
PORT	1883	Default MQTT port
TOPIC	lab/dsa	Example topic name (customisable)

3. Publisher (publisher.py)

The publisher script connects to the broker and sends a random number to a topic at a fixed interval. Complete the TODO sections using the hints provided.

3.1 Role of the Publisher

- Connects to the MQTT broker on startup.
- Runs a continuous loop, generating a random value every INTERVAL seconds.
- Publishes each value to the configured topic.
- Prints each value to the console before publishing.

3.2 Complete the Code

Fill in the configuration variables at the top of the file with the broker address, port, and topic you set up in Section 2.

```
import paho.mqtt.client as mqtt
import time

# --- Config ---
BROKER = "" # e.g. "localhost"
PORT = 0 # e.g. 1883
TOPIC = "" # e.g. "lab/sensor"
INTERVAL = 10

# --- MQTT Client ---
# TODO 1: Instantiate the MQTT client
client = None

# TODO 2: Connect to the broker

# TODO 3: Start the network loop

# --- Publish loop ---
```

```

while True:

    # TODO 4: Generate a random number between 0 and 100
    value = None

    # TODO 5: Print the value before publishing

    # TODO 6: Publish the value to TOPIC

    time.sleep(INTERVAL)

```

Step-by-step hints

TODO 1 – Instantiate client	<i>client = mqtt.Client()</i>
TODO 2 – Connect to broker	<i>client.connect(BROKER, PORT, 60)</i>
TODO 3 – Start network loop	<i>client.loop_start()</i>
TODO 4 – Random number	<i>import random then value = random.randint(0, 100)</i>
TODO 5 – Print value	<i>print(f"Publishing: {value}")</i>
TODO 6 – Publish	<i>client.publish(TOPIC, value)</i>

3.3 Running the Publisher

```
python publisher.py
```

You should see a line printed every INTERVAL seconds, for example:

```

Publishing: 42
Publishing: 87
Publishing: 15

```

4. Subscriber (subscriber.py)

The subscriber script listens on the same topic as the publisher and prints every message it receives. It relies on callbacks that are triggered automatically by the Paho library.

4.1 Role of the Subscriber

- Connects to the same MQTT broker and subscribes to the same topic as the publisher.
- Runs a blocking loop that keeps the script alive and processes incoming messages.
- Prints a confirmation when the connection is established.
- Prints the topic and payload of every message received.

4.2 Complete the Code

Use the same BROKER, PORT, and TOPIC values as in pub.py. Both scripts must share the same topic for communication to work.

```
import paho.mqtt.client as mqtt

# --- Config ---
BROKER = "" # same as publisher
PORT = 0 # same as publisher
TOPIC = "" # same as publisher

# --- Callbacks ---
def on_connect(client, userdata, flags, rc):
    # TODO 1: Print a confirmation if rc == 0

    # TODO 2: Subscribe to TOPIC
    pass

def on_message(client, userdata, msg):
    # TODO 3: Decode the received payload
    value = None

    # TODO 4: Print topic and value
    pass

# --- MQTT Client setup ---
# TODO 5: Instantiate the MQTT client
client = None

# TODO 6: Assign callbacks to the client

# TODO 7: Connect to the broker

# TODO 8: Start blocking loop
```

Step-by-step hints

TODO 1 – Check connection	<i>if rc == 0: print("Connected successfully")</i>
TODO 2 – Subscribe	<i>client.subscribe(TOPIC)</i>
TODO 3 – Decode payload	<i>value = msg.payload.decode()</i>
TODO 4 – Print message	<i>print(f"[{msg.topic}] {value}")</i>
TODO 5 – Instantiate client	<i>client = mqtt.Client()</i>
TODO 6 – Assign callbacks	<i>client.on_connect = on_connect + client.on_message = on_message</i>
TODO 7 – Connect	<i>client.connect(BROKER, PORT, 60)</i>
TODO 8 – Blocking loop	<i>client.loop_forever()</i>

4.3 Running the Subscriber

Open a second terminal and run the subscriber while the publisher is already running:

```
python subscriber.py
```

Expected output (messages arrive every INTERVAL seconds):

Connected successfully
[lab/dsa] 42
[lab/dsa] 87
[lab/dsa] 15

4.4 Key Difference: loop_forever vs loop_start

Method	Behaviour	Used in
loop_forever()	Blocks execution – script stays alive waiting for messages.	Subscriber
loop_start()	Non-blocking – network runs in background thread; script continues.	Publisher